

Seismic Blind Deconvolution Based on Self-Supervised Machine Learning (SSL-BD)

Applied Sciences, 2024 - mdpi.com

Xia Yin, Wenhao Xu, Zhifang Yang, Bangyu Wu

4 de setembro de 2026

Métodos Avançados de Inteligência Artificial para Processamento de Dados Sísmicos
COPPE - UFRJ

Sumário

Problema

BBL-BD

Arquitetura

Etapa 1

Etapa 2

Etapa 3

Experimentos

Resultados

Input: IN, Target: RFLT

Resultados

Input: IMG

Problema

Problema

A ideia básica da deconvolução sísmica é:

$$y = w * r \quad (1)$$

onde:

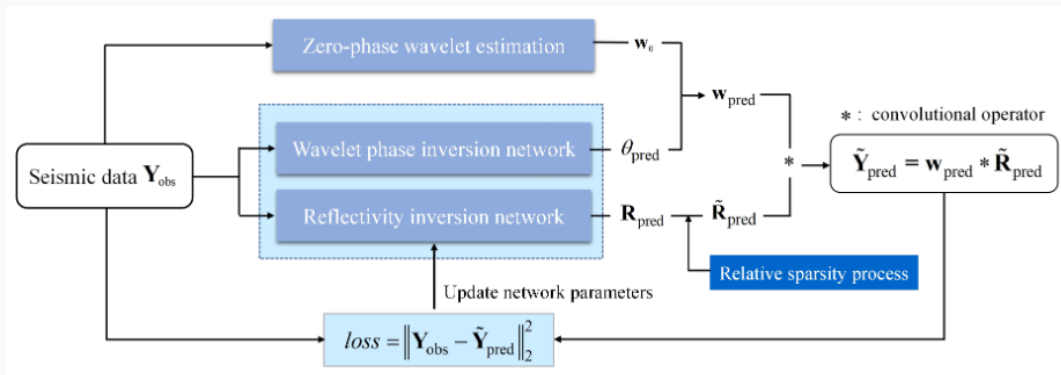
- y : dado sísmico observado;
- w : wavelet sísmica;
- r : refletividade;
- $*$: convolução

Blind deconvolution é um problema mal-posto e subdeterminado em que w e r são desconhecidos

BBL-BD

BBL-BD

A ideia do BBL-BD consiste em usar o dado observado para estimar o wavelet e a refletividade usando duas redes neurais combinadas



Arquitetura

Etapa 1

O método não começa a rede com uma wavelet completamente aleatória.

Ele tenta obter uma estimativa inicial da wavelet a partir do próprio dado.

Passos:

1. Cálculo da média dos traces $\bar{y} = \frac{1}{N_{traces}} \sum_i y_i$
2. Cálculo da $FFT(\bar{y})$ e espectro de amplitude $|FFT(\bar{y})|$
3. Suavização do espectro de amplitude com um polinômio cúbico local de 7 pontos (filtro de Savitzky-Golay)
4. Aplicação da IFFT ao espectro suavizado

A wavelet inicial (w_0) é calculada como $IFFT(smooth(|FFT(\bar{y})|))$

Etapa 2

A próxima etapa estima a fase θ

$$\theta_{pred} = NN_{\theta}(Y_{obs}; \phi_{\theta}) \quad (2)$$

A arquitetura possui:

- 4 camadas convolucionais;
- 2 fully connected layers.

A primeira convolução usa kernel 29×1 e stride 14.

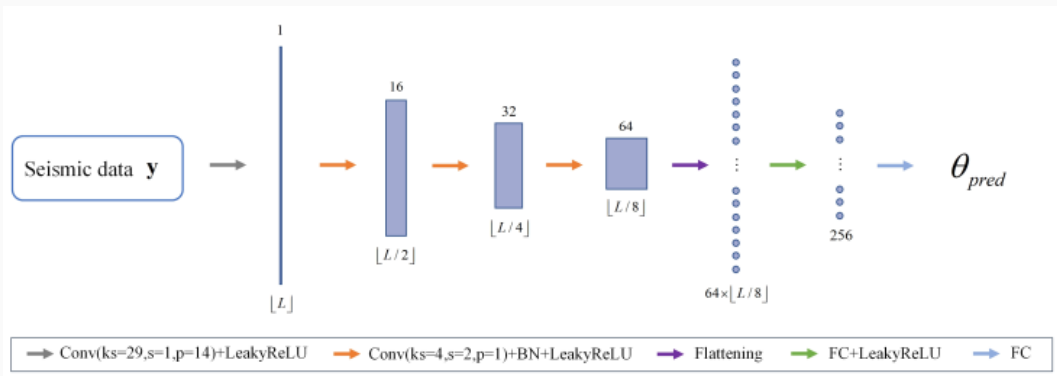
As seguintes usam 4×1 e stride 2.

Depois ocorre flattening e duas camadas fully connected, resultando em um único valor de fase.

O método usa a transformada de Hilbert para girar w_0 no domínio de fase por θ

$$w_{pred} = Re[(w_0 + jHilbert(w_0))e^{-j\theta_{pred}}] \quad (3)$$

Etapla 2



Etapa 3

Por último, método utiliza uma Residual Neural Network para prever a refletividade:

$$R_{pred} = NN_R(Y_{obs}; \phi_R) \quad (4)$$

onde ϕ_R são os parâmetros da rede.

A arquitetura possui:

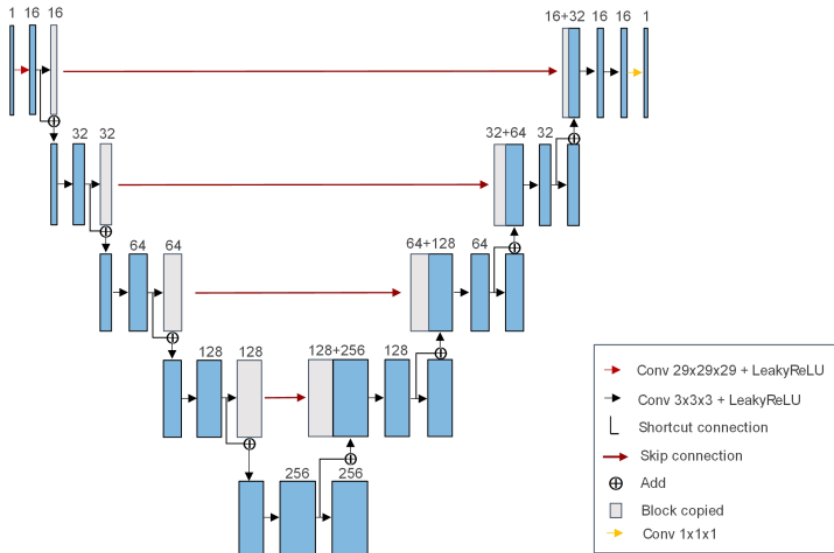
- residual blocks;
- convoluções;
- LeakyReLU;
- long skip connections.

Para o primeiro bloco, o kernel é $29 \times 29 \times 29$ no caso 3D.

Os demais usam $3 \times 3 \times 3$, e a última camada utiliza $1 \times 1 \times 1$

Para o caso 2D, eles simplesmente substituem as convoluções 3D por convoluções 2D.

Etapa 2



Sumarização do método

O método emprega uma restrição de sparsity relativa, em que o threshold (μ) de sparsity aumenta durante o treinamento com base em $r_{epoch} = \frac{\text{época atual}}{\text{total de épocas}}$

1. Rede recebe Y_{obs}
2. Estima-se wavelet de fase-zero
3. Rede produz θ_{pred} e R_{pred}
4. Com θ_{pred} , calcula-se w_{pred}
5. Aplica-se sparsity $R_{pred} \rightarrow \bar{R}_{pred}$
6. Faz-se a operação física $\bar{Y}_{pred} = w_{pred} * \bar{R}_{pred}$
7. A perda é calculada como $\mathcal{L} = ||Y_{obs} - \bar{Y}_{pred}||_2^2$

Experimentos

Experimentos

Dados:

- Input: IN, Target: RFLT
- Input: IMG

Paradigma:

- Self-Supervised ($\mathcal{L} = ||Y_{obs} - \bar{Y}_{pred}||_2^2$)
- Supervised ($\mathcal{L} = ||R - \bar{R}_{pred}||_2^2$)

Feature maps da RNN

- $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024 \rightarrow 256 \rightarrow 512 \rightarrow 128 \rightarrow 64$

Resultados

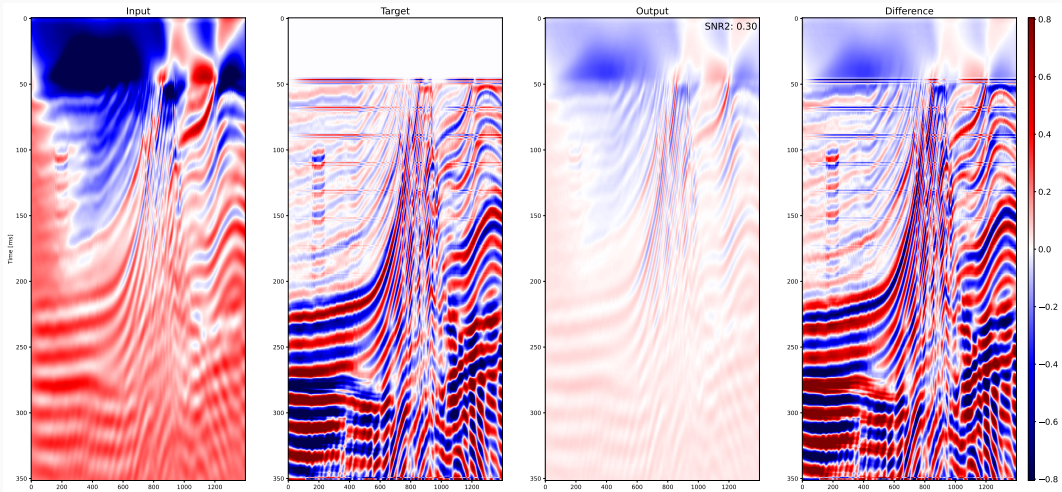
Input: IN, Target: RFLT

Resultados

Seismic	Problem	Method	PSNR	SNR2
IN-RFLT	Deblurring	SSL-BD (self-supervised)	15.03	0.30
		SSL-BD (supervised)	28.41	0.85

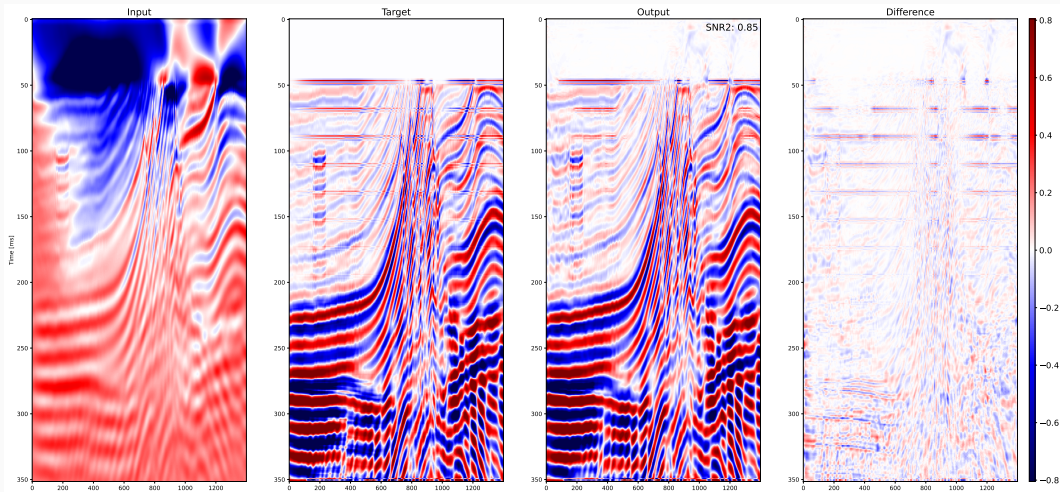
Self-Supervised

Treinado com IN



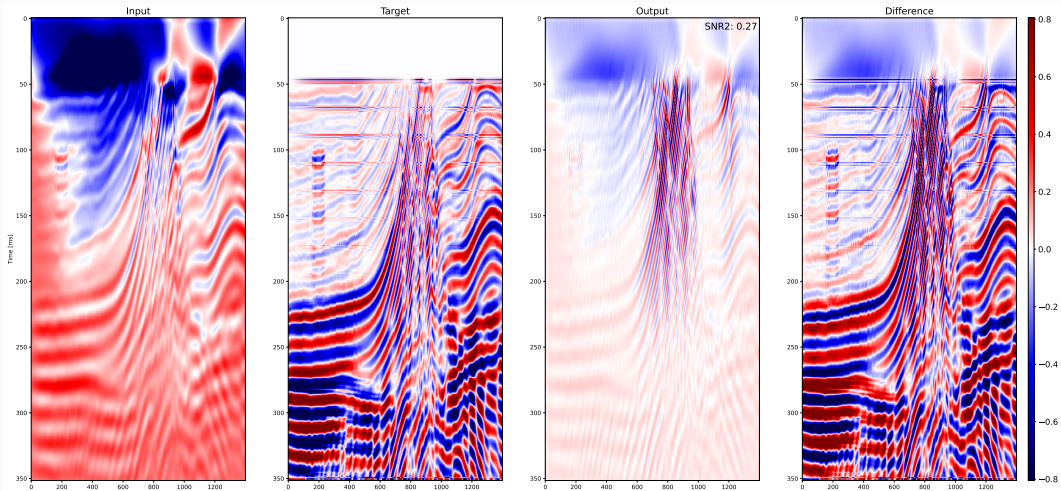
Supervised

Treinado com IN-RFLT



Fine-tuning do Self-Supervised com wavelet do Supervised

Treinado com IN

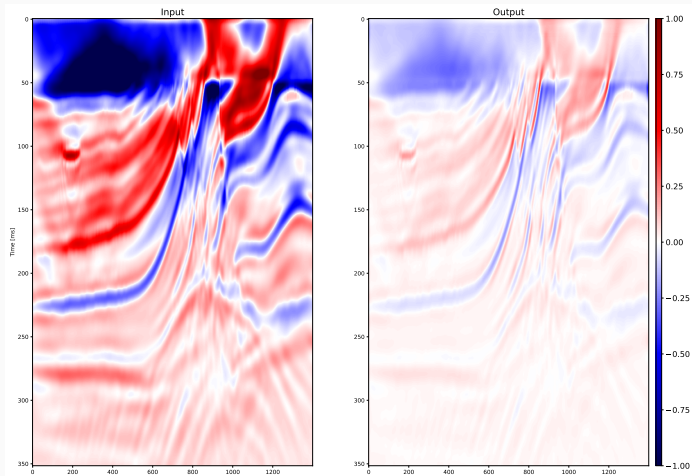


Resultados

Input: IMG

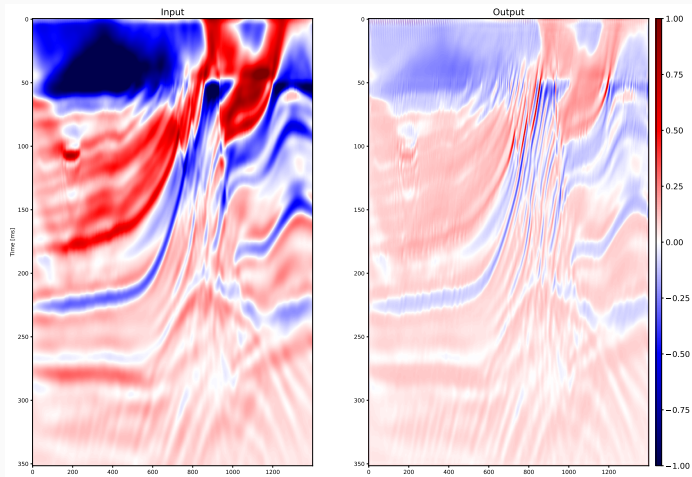
Self-Supervised

Treinado com IMG



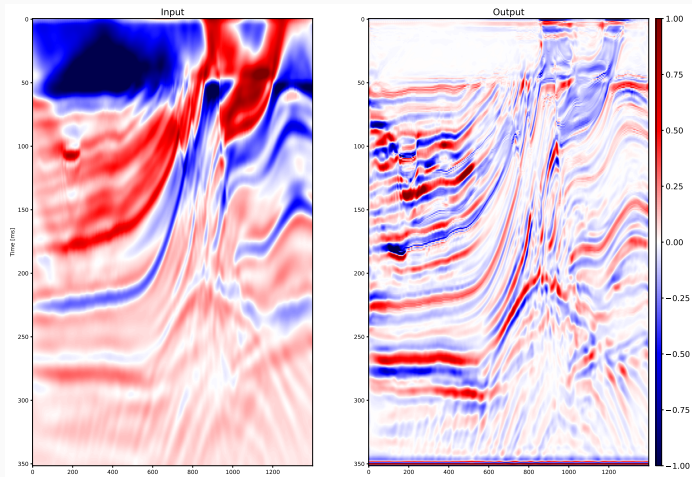
Self-Supervised

Treinado com IN



Supervised

Treinado com IN-RFLT



Self-Supervised com o output do Supervised

Treinado com output do teste do Supervised treinado com IN-RFLT

